

Architectural Decision Record:

Reactive Data Access Layer

This document details the selection of Lettuce as the Redis driver for the Sentinel project, supporting 10,000+ concurrent users.

1. The Challenge: Concurrency in Reactive Systems

Traditional Redis drivers (e.g., Jedis) are **blocking**. In a high-throughput WebFlux application, if a thread blocks while waiting for Redis, it starves the Netty event loop, limiting total system throughput to the number of available threads.

2. Why Lettuce?

Lettuce's non-blocking, Netty-based architecture is the only choice that maintains the thread-safety and latency guarantees required for high-throughput WebSocket systems.

Lettuce is built on **Netty**, an asynchronous, event-driven networking library. This provides several critical advantages:

- **Non-blocking I/O:** When an operation is sent to Redis, the calling thread is immediately freed to handle other requests.
- **Single Connection Multiplexing:** Lettuce uses a single, thread-safe connection to Redis that multiple threads can share, reducing overhead.
- **Pipeline Efficiency:** It naturally supports pipelining, drastically reducing network round-trips for high-frequency operations like rate limiting.

3. Supporting 10,000 Concurrent Users

At 10,000 users, the overhead of context switching and blocked threads is the primary bottleneck. By moving to a fully reactive Lettuce-based stack:

- **Event-Loop Efficiency:** A small number of Event Loop threads (usually proportional to CPU cores) can manage thousands of concurrent I/O operations without being blocked.
- **Deterministic Latency:** Because operations don't queue behind blocked threads, latency remains consistent, preventing "tail latency" spikes during traffic bursts.